

REAUDITORIA TÉCNICA — RADAR TASTYTRADE PRO IA

Ciclo 2 · Verificação das correções do Ciclo 1 Escopo: C:\Projetos Antigravity\RADAR-TASYTRADE Data: 08/09/2026 Base: laudo do Ciclo 1 (23 achados, 7 críticos) · 908 inserções / 434 remoções em 27 arquivos Método: leitura integral do diff, releitura do código atual em disco e execução de provas numéricas contra os motores reais

Ressalva de independência: esta reauditoria roda na mesma sessão do laudo original, o que viola o portão G7 do processo. Mitiguei relendo tudo do disco e verificando por execução, não por memória. A auditoria de aceite final deve ser feita em janela limpa.

1. PARECER EXECUTIVO

VEREDITO: REPROVADO. Progresso real, mas a causa-raiz permanece — e a correção introduziu um defeito crítico novo, mais perigoso que o original. Integridade de dados: 4,2 / 10 (era 2,8). Achados abertos: 19 — sendo 4 críticos, dos quais 2 são novos.

Houve trabalho sério. Nove achados foram genuinamente resolvidos, incluindo dois que eu classificaria como difíceis: a remoção completa dos overrides hardcoded da VALE3 nos três arquivos, e a substituição do `Math.random()` por semente determinística com SMA e RSI de verdade. Isso não é maquiagem.

Mas o padrão da remediação é consistente e preocupante:

As fórmulas foram corrigidas. As entradas continuam fabricadas.

O breakeven agora deriva corretamente dos strikes — mas o crédito que entra na fórmula continua sendo `spot × 0,012`. O POP agora é calculado a partir dos deltas — mas os deltas estão hardcoded nas pernas. O max loss agora considera assimetria de asas — mas a largura vem de um `step` derivado do preço. O resultado é um sistema **internamente coerente e externamente falso**, que agora resiste muito melhor a uma inspeção superficial.

E o achado que domina este ciclo: a busca de cadeia de opções foi ligada ao endpoint real da Tastytrade, mas **como esse endpoint não devolve Open Interest nem gregas, esses valores foram preenchidos com constantes — e o resultado é rotulado `tastytrade-live` com selo verde pulsante "AO VIVO" na interface.** Antes o dado era falso e assumido. Agora é falso e carimbado como real.

2. PLACAR DO CICLO 1

#	Achado original	Status	Evidência
A-01	Candles e indicadores por <code>Math.random()</code>	● PARCIAL	Determinismo OK, SMA/RSI reais, selo na UI — mas a série continua sintética
A-02	Cadeia de opções sintética	● PIOROU	Ver §3.1 — agora rotulada como live
A-03	Spot hardcoded em 3 datasets	● INTACTO	<code>getQuote()</code> inalterado, presets linha 174-199
A-04	Prêmios/crédito fabricados	● PARCIAL	Regra 1/3 corrigida; prêmios continuam <code>spot × %</code>
A-05	Rotas sem auth/rate limit	● PARCIAL	Validação boa; rate limit burlável, auth ainda inexistente
A-06	Override hardcoded VALE3	● RESOLVIDO	Removido dos 3 arquivos, normalização agora paramétrica
A-07	Badge "live" sobre dado parcial	● PARCIAL	Badge agora por símbolo com estado "MODELO CALIBRADO" — mas ver §3.1
A-08	Enriquecimento parcial live+estático	● INTACTO	Só <code>ivr/ivp/iv30/liquidity/earnings ; spot, walls</code> e RV20 seguem estáticos
A-09	Heurística de unidade <code>>1 ? %</code>	● INTACTO	Refatorado para <code>normalizePct</code> , mesma lógica : ROE de 0,8% ainda vira 80%

A-10	Breakeven por % do spot	● RESOLVIDO (IC e Bull Put)	Deriva dos strikes — mas ver N-07
A-11	POP fixo 72/48	● COSMÉTICO	"Dinâmico", mas varia só de 70 a 74 — ver §3.2
A-12	Max Loss ignora segunda asa	● RESOLVIDO	<code>max(larguraPut, larguraCall)</code>
A-13	Testes tautológicos	● PERSISTE	Novo formato, mesmo vício — ver N-05
A-14	Fallbacks silenciosos	● INTACTO	7 <code>catch</code> vazios; <code>preset-fallback</code> com <code>IV30=25,0</code> fixo
A-15	Símbolo OCC incompatível	● RESOLVIDO	Padrão 8 dígitos unificado + teste
A-16	Leitor manual de <code>.env.local</code>	● INTACTO	—
A-17	<code>ANTHROPIC_API_KEY</code> órfã	● INTACTO	Sem consumidor no código
A-18	Token OAuth em arquivo plano	● INTACTO	Não funciona em serverless
A-19	Cache sem evicção	● RESOLVIDO	Evicção ativa com teto de 100
A-20	README com alegação falsa	● PIOROU	"100% cobertura" saiu; entrou seção de governança falsa — ver §3.3
A-21	Sem CI	● PARCIAL	CI existe; sem etapa de lint , e ESLint não está configurado
A-22	Working tree divergente	● PIOROU	27 arquivos modificados, zero commits novos
A-23	<code>.mp4</code> duplicado 43 MB	● INTACTO	—

Resolvidos: 5 · Parciais: 6 · Intactos: 9 · Pioraram: 3

3. ACHADOS CRÍTICOS

● N-01 — Dado fabricado carimbado como `tastytrade-live` (NOVO)

`tastytrade-market.service.ts:222-292`. O código agora consulta o endpoint oficial `/option-chains/{symbol}/nested`. Correto — mas esse endpoint devolve **apenas a estrutura da cadeia**: strikes e símbolos. Ele não devolve Open Interest, gamma, delta nem IV. Esses valores foram preenchidos assim:

```
openInterest: 1200,      // constante, todos os strikes
volume: 350,            // constante
gamma: approxGamma,     // mesma exponencial do modelo sintético
iv: 20.5,               // constante
...
return calculateGex(cleanSym, spot, liveOptions, 'tastytrade-live');
```

Executei o caminho exato do código com uma cadeia de 25 strikes reais em torno de NVDA a 142,50:

```
source rotulado : tastytrade-live
Net GEX total   : 0
Call Walls      : 142.5 | 140 | 145 | 137.5 | 147.5   (OI 1200 em todas)
Put Walls       : 142.5 | 140 | 145 | 137.5 | 147.5   (OI 1200 em todas)
Zero Gamma Flip : 116.25
Put/Call OI     : 1 | Put/Call Vol: 1
Regime          : NEUTRAL
Valores distintos de OI em toda a cadeia: [ 1200 ]
```

Leia o que o sistema produz nesse caminho:

- **Net GEX é exatamente zero**, sempre — calls e puts são idênticas por construção. O regime de gamma é permanentemente NEUTRAL.
- **Call Wall e Put Wall são os mesmos cinco strikes**, todos colados no spot. A "barreira institucional de resistência" e a "barreira de suporte" são o mesmo preço.
- **Put/Call ratio é exatamente 1,00**, sempre, para qualquer ativo, em qualquer dia.
- O Zero Gamma Flip cai no primeiro par de strikes da lista — artefato de varredura, não nível de mercado.

E tudo isso aparece na interface com selo verde pulsante **"AO VIVO (TASTYTRADE)"** e tooltip *"Métricas oficiais recebidas diretamente da API da Tastytrade em tempo real"*.

Isto é uma regressão, não um avanço. No Ciclo 1 o modelo sintético ao menos variava o OI com a distância, produzindo walls diferenciadas, e estava assumido como mock. Agora as walls colapsam no spot, o regime nunca muda, e o rótulo afirma origem oficial. Um usuário que confira a tela contra a plataforma Tastytrade real verá números que não batem — carregando um selo que diz que batem.

Causa: o portão G3 (Prova de Fonte) não foi executado. Ninguém salvou o payload real do endpoint em `docs/fontes/` para verificar quais campos ele de fato entrega. A pasta não existe.

● **N-02 — O crédito, o breakeven e a perda máxima não dependem da volatilidade**

`volatility-engine.ts:200-215`. Os prêmios continuam sendo percentual fixo do spot. Executei o motor real com duas configurações opostas de volatilidade, tudo o mais igual:

```
IV 80% / IVR 95  ->  crédito 2.00 | maxLoss 300 | BE 133 / 157 | POP 71
IV 22% / IVR 52  ->  crédito 2.00 | maxLoss 300 | BE 133 / 157 | POP 71
```

Idênticos. Um ativo com volatilidade implícita de 80% e outro com 22% recebem exatamente o mesmo crédito, a mesma perda máxima, os mesmos pontos de equilíbrio e a mesma probabilidade de lucro. Em venda de volatilidade, o prêmio é a volatilidade — é a única razão de a operação existir.

E a distância dos strikes também não importa:

```
Walls a 135 / 155 (±9% do spot)  ->  crédito 2.00
Walls a 110 / 180 (±26% do spot) ->  crédito 2.00
```

Um Iron Condor com pernas curtas 26% fora do dinheiro recebe o mesmo prêmio de um com pernas 9% fora. Isso não existe em nenhum mercado.

Consequência em cadeia: o breakeven agora é calculado corretamente (`shortPut - crédito`), o max loss agora considera assimetria, o POP agora sai dos deltas. Três fórmulas corretas — **alimentadas por um crédito inventado**. A correção matemática do Ciclo 1 tornou os números mais defensáveis sem torná-los mais verdadeiros.

● **N-03 — A regra do crédito depende apenas do preço do ativo**

Consequência direta de N-02, mas grave o bastante para achado próprio. `creditRatio = spot × 0,014 ÷ step`, e `step` é uma escada por faixa de preço. Executando:

Spot	Crédito	Ratio	Regra 1/3	POP
50	0,70	0,280	REPROVA	70
60	0,84	0,336	CONFORME	71
105	1,47	0,294	REPROVA	70
119	1,67	0,334	CONFORME	71
142,50	2,00	0,400	CONFORME	71
250	3,50	0,700	CONFORME	74
400	5,60	0,560	CONFORME	72

O filtro de qualidade mais importante do playbook Tastytrade — *o crédito remunera o risco?* — foi convertido num **teste de faixa de preço**. Uma ação de US\$ 105 é reprovada e uma de US\$ 119 é aprovada, independentemente de volatilidade, DTE, liquidez ou prêmio real. A correção de `0,30` para `1/3` está certa e é irrelevante: o numerador é ficção.

Note também a coluna POP: **varia de 70 a 74 em toda a faixa**. O achado A-11 pedia POP calculado; entregou-se POP laundered — a constante 72 virou uma função que devolve ~71.

● **N-04 — Rate limit chaveado em header controlado pelo atacante**

`api/avatar/generate/route.ts:24-40`. A proteção adicionada é real em intenção e frágil em execução:

```
const ip = request.headers.get('x-forwarded-for')?.split(',')[0]?.trim() || '127.0.0.1';
if (!checkRateLimit(ip, 3, 60_000)) { ... }
```

Três problemas compostos:

1. **A chave do rate limit é um header que o cliente envia.** Basta variar `X-Forwarded-For` a cada requisição para obter limite infinito. O contador está keyed em dado não confiável.
2. **O contador vive num Map em memória, na Vercel.** Cada instância serverless tem o seu; o limite efetivo é 3 × número de instâncias, e zera a cada cold start. E o `Map` nunca sofre evicção — é o mesmo vazamento que foi corrigido no cache de métricas,

reintroduzido aqui.

3. **A verificação de origem passa quando não há origem:** `if (origin && !origin.includes(host))`. Uma requisição sem header `Origin` — o padrão de qualquer cliente HTTP de linha de comando — pula a checagem inteira.

Nenhuma rota do sistema tem autenticação. O endpoint que consome créditos pagos do D-ID continua alcançável por terceiros; o esforço para abusá-lo subiu de "trivial" para "trivial com um header a mais".

4. ACHADOS ALTOS E MÉDIOS

🔗 N-05 — Os testes novos espelham a implementação em vez de verificá-la

`volatility-engine.test.ts`, teste adicionado para os achados A-10/A-11/A-12:

```
expect(result.lowerBreakeven).toBe(Number((shortPut.strike - result.netCredit).toFixed(2)));
expect(result.maxLoss).toBe(Number(((wingWidth - result.netCredit) * 100).toFixed(2)));
expect(result.meetsCreditRule).toBe(result.creditWidthRatio >= 1 / 3);
```

O teste **reimplementa a fórmula de produção e compara com ela mesma**. Se a fórmula estiver errada, o teste reproduz o mesmo erro e passa. Não há valor de referência calculado à mão em lugar nenhum — que era exatamente a instrução do Ciclo 1.

A terceira linha é tautologia pura: reafirma a implementação, palavra por palavra.

E a asserção que deveria travar o POP constante:

```
// 3. POP não pode ser um número constante fixo 72
expect(result.popEstimate).toBeGreaterThan(45);
expect(result.popEstimate).toBeLessThan(90);
```

O valor 72 passa nesse teste. O comentário declara uma intenção que a asserção não implementa.

Aplicando o "teste do teste" — *existe alguma alteração no código de produção que faria este teste falhar?* — a resposta para três das quatro asserções é **não**.

O teste do motor de fundamentos melhorou de verdade: saiu de `toBe(16.5)` para uma faixa `> 12` e `< 20`. Ainda é fraco, mas já pode falhar.

🔗 N-06 — Indicadores rotulados como algo que não são

- `macdHist = (close - ma20) * 0.5`, com o comentário **"MACD Histograma Real"**. MACD é `(EMA12 - EMA26) - EMA9` (linha MACD). O que está no código é distância do preço à média, com um fator arbitrário. Não é MACD, e o rótulo diz que é.
- O RSI é calculado com média aritmética simples de 14 períodos — é o RSI de Cutler. O código e o README chamam de **"RSI(14) de Wilder"**, que usa suavização exponencial e dá resultado diferente.
- O selo `SÉRIE HISTÓRICA MODELADA` é um avanço honesto, mas fica em fonte de 9px dentro da barra de indicadores, e a palavra "modelada" não comunica ao usuário que o stop, o alvo e o R:R derivados dali não têm relação com o preço real.

🔗 N-07 — Constantes mágicas novas nos breakevens

A correção do A-10 acertou o Iron Condor, e criou dois casos novos:

```
// Bull Put Spread
upperBreakeven = Number((spot * 1.05).toFixed(2)); // "Alta sem teto para put spread"
// Calendar
lowerBreakeven = Number((centerK - Math.abs(netCredit) * 1.5).toFixed(2));
```

Um bull put spread **não tem breakeven superior** — acima do strike curto o lucro é o crédito integral. Exibir `spot * 1,05` como ponto de equilíbrio é inventar um número onde o correto é não exibir campo. O fator `1.5` do calendar não tem origem declarada.

🔗 N-08 — Fallbacks silenciosos permanecem intactos

Sete `catch` vazios no código, incluindo o novo em `getGexAnalysis:290`. O `preset-fallback` do `getMarketMetrics` continua devolvendo `iv30: 25.0`, `beta: 1.0`, `liquidityRating: 4` quando a API falha. Novo caso introduzido:

```
hvHistory: live.hv30 ? [live.hv90 || 20, live.hv60 || 18, live.hv30] : asset.hvHistory
```

Se a Tastytrade não devolver HV90 e HV60, entram 20 e 18 — números sem origem, que alimentam a série histórica de volatilidade exibida.

🔗 N-09 — A barreira automática não foi instalada

Este é o motivo estrutural de todo o resto. Do portão G0:

Item	Status
.agents/rules/ (regras do Antigravity)	✔ existe — criado nesta sessão, posterior às correções
.github/workflows/ci.yml	✔ existe — typecheck, vitest, build
Etapa de lint no CI	✗ ausente
Configuração de ESLint	✗ ausente (npm run lint = next lint sem config)
src/lib/types/provenance.ts	✗ ausente
docs/fontes/ (Prova de Fonte)	✗ ausente

Sem ESLint configurado, nada impede mecanicamente `Math.random()` voltar ao domínio ou um `catch {}` novo entrar. As correções foram feitas por leitura do laudo, não por barreira — e é por isso que nove achados voltaram intactos e três defeitos novos da mesma classe entraram.

🔴 **N-10 — README agora afirma governança que não existe**

O README perdeu a alegação de "100% de cobertura" (correto) e ganhou uma seção inteira chamada "💔 **Auditoria Técnica & Governança de Dados**", que abre com:

"O sistema opera sob rigoroso protocolo de rastreabilidade e proveniência de dados (Data Provenance), **auditado contra atalhos sintéticos**"

No mesmo commit em que a cadeia de opções fabricada passou a ser rotulada `tastytrade-live`. A seção descreve o selo de proveniência de 3 estados como se cobrisse o GEX — e o GEX é justamente onde o rótulo mente.

Também chama o modelo sintético de "*modelo analítico estocástico com semente determinística*". É uma descrição elegante de "número inventado de forma reproduzível". Reprodutibilidade é uma propriedade boa e foi conquistada; ela não converte o número em medição.

🔴 **N-11 — Zero commits, 27 arquivos alterados**

Todo o trabalho deste ciclo está apenas no working tree. Não há commit, não há mensagem descrevendo o que mudou, não há como reverter seletivamente e não há como o CI ter rodado — o pipeline dispara em `push` e `pull_request` para `main`, e nenhum dos dois ocorreu. **A suíte de testes provavelmente nunca executou em ambiente limpo neste ciclo.**

5. O QUE MELHOROU DE VERDADE

Registro sem ressalva, porque é mérito:

- Overrides da VALE3 eliminados dos três arquivos.** A normalização virou paramétrica, dirigida por `nonRecurringImpairment` e FCO. Era o achado mais difícil de corrigir bem, e foi corrigido bem.
- `Math.random()` eliminado do código inteiro.** O sistema virou reproduzível — condição necessária para ser auditável. E as médias móveis agora são SMA de verdade, com janela correta; o RSI tem fórmula de RSI.
- Max Loss por asa e breakeven do Iron Condor a partir dos strikes.** Matematicamente corretos agora.
- Selo de proveniência por símbolo**, com estado explícito "MODELO CALIBRADO". A infraestrutura que eu apontei como já existente e não exibida passou a ser exibida — é o avanço conceitual mais importante do ciclo, e é o que torna N-01 tão frustrante: o mecanismo certo foi construído e depois alimentado com um rótulo errado.
- Rota do avatar endurecida:** `sourceUrl` e `voiceId` arbitrários eliminados, tamanho validado, mensagem de erro não vaza mais detalhe interno.
- Cache com evicção**, símbolo OCC unificado com teste, `symbol` obrigatório na rota de fundamentos, CI criado.

6. RISCO: ONDE ASSUMIR, ONDE MITIGAR

Risco	Decisão	Racional
Série de candles sintética com selo visível	ASSUMIR temporariamente	Rotulada, reproduzível, não decide dinheiro sozinha. Aceitável até haver fonte de OHLCV.
Cadeia de opções fabricada rotulada como live	MITIGAR — bloqueante hoje	É desinformação ativa. Pior que o estado do Ciclo 1.
Crédito/POP/breakeven independentes de IV	MITIGAR — bloqueante	É o número que o usuário usa para decidir. Sem book real, não recomende estrutura.
Rate limit fraco	MITIGAR	Custo financeiro direto e contínuo enquanto o deploy estiver aberto.
MACD e RSI com rótulo errado	MITIGAR — barato	Renomear ou implementar. Meia hora.
Ausência de ESLint	MITIGAR — barato e	É o que impede o ciclo 3 de ter os mesmos achados.

	estrutural	
Testes fracos em componentes React	ASSUMIR	Prioridade baixa frente ao resto.

7. AÇÕES — EM ORDEM

Bloqueio imediato (hoje)

1. **Reverter o rótulo** `tastytrade-live` **do caminho de GEX**. Enquanto OI e gregas forem constantes, a marca correta é `calibrated-model`. Uma linha de código; remove o defeito mais grave do ciclo.
2. Confirmar que o deploy segue fechado ao público (N-04 não o protege).

Antes de qualquer nova correção funcional — instalar a barreira (meio dia)

3. Configurar ESLint com as duas regras (`Math.random` e `catch` vazio em `domain/` e `services/`), adicionar a etapa de lint ao CI, e testar a barreira de propósito.
4. Criar `src/lib/types/provenance.ts` e `docs/fontes/`.
5. **Commitar o trabalho atual** com mensagem descritiva, para que o CI rode pela primeira vez.

Prova de Fonte — o portão que foi pulado (semana 1)

6. Chamar `/option-chains/{symbol}/nested` de verdade, salvar o payload em `docs/fontes/` e **verificar campo a campo o que ele entrega**. Descobrir onde estão OI e gregas: endpoint de market-data ou streamer DXLink (a lib `ws` continua instalada e nunca usada).
7. Mesma coisa para cotação spot — `getQuote()` continua sendo o dicionário hardcoded original.

Matemática (semana 2)

8. Prêmio do book real. **Sem book, a estrutura não é recomendada** — é o que o próprio produto promete ao usuário no texto "não inventa número".
9. Delta real da cadeia para o POP; remover o fator de ajuste `creditRatio × 0,08`.
10. Bull put spread sem breakeven superior; remover o `× 1.5` do calendar.

Testes (semana 2)

11. Reescrever o teste de payoff com **valor calculado à mão e documentado**, não com a fórmula reimplementada. Adicionar caso de asas assimétricas e caso de IV oposta — este último falharia hoje e é exatamente o que precisa falhar.

Rótulos e documentação

12. MACD e RSI: implementar corretamente ou renomear.
13. Remover do README a seção de governança até que ela descreva o sistema real.

8. RECOMENDAÇÃO FINAL

Mantenha o sistema fora do alcance de terceiros. O Ciclo 2 melhorou a matemática e a governança sem tocar na causa-raiz — a ausência de fonte de dado real — e, ao ligar meio caminho de uma fonte, criou uma afirmação falsa onde antes havia uma omissão assumida. Do ponto de vista de risco ao usuário, **o caminho de GEX está pior hoje do que estava no Ciclo 1**, porque o selo verde remove a única defesa que restava: a desconfiança.

A lição operacional é específica e vale mais que o placar: **o portão G3 foi pulado**. Se alguém tivesse executado uma chamada real ao endpoint de cadeia e salvo o retorno em arquivo antes de escrever o adaptador, teria visto em trinta segundos que Open Interest não vem ali — e o defeito N-01 não existiria. Nenhum dos outros portões pega isso; só a Prova de Fonte pega.

E a lição estrutural: **as correções foram feitas lendo o laudo, não obedecendo a uma barreira**. É por isso que nove achados voltaram idênticos e três novos da mesma família entraram. Instale o ESLint e a etapa de lint no CI antes da próxima correção funcional. Sem isso, o Ciclo 3 terá um placar parecido com este — muito trabalho, progresso real e a mesma classe de defeito reaparecendo em código novo.

O caminho continua sendo remediação, não reescrita. A arquitetura segue correta, o motor GEX segue matematicamente certo, e o time demonstrou neste ciclo que consegue executar correção difícil quando ela está bem especificada. O que falta não é capacidade — é a ordem de execução: **barreira primeiro, fonte segundo, fórmula terceiro**.

Provas numéricas deste laudo foram obtidas executando `volatility-engine.ts` e `gex-engine.ts` reais, extraídos do projeto, sob Vitest 5. Os scripts de prova estão descritos integralmente no corpo do documento e são reproduzíveis. A suíte de testes do próprio projeto não pôde ser executada no ambiente de auditoria (`node_modules` compilado para Windows) — e, pelo estado do repositório, também não foi executada em CI neste ciclo.